MLOps Lab Assignment 2

Tracking a Pre-trained Sentiment Model with MLflow

Name	Jeevitha V
Roll No. (USN)	4GW23CI020
Experiment	sentiment-inference-lab
Model	distilbert-base-uncased-finetuned-sst-2-english (pre-trained, inference only)
Tools	Python 3.13.2, MLflow 3.17.2, Hugging Face Transformers, PyTorch, pandas
Platform	Windows 11, Command Prompt, virtual environment *mlops-env*

1. Objective

In this lab I used MLflow to keep a record of an inference-only experiment. A ready-made Hugging Face sentiment model (DistilBERT fine-tuned on SST-2) classifies ten short labelled sentences. For every run, MLflow stores the settings used, the results obtained and a predictions file. The same model was run with three batch sizes (1, 8 and 4), and the runs were then compared in the MLflow UI to see how batch size affects speed and accuracy. No model training was done.

2. Environment Setup

The setup followed the steps in the assignment sheet. Every command was typed in Command Prompt. Figure 1 shows the version checks, the virtual environment and the package installation.

Step	Command	Outcome
1	python --version pip --version	Python 3.13.2 and pip 24.3.1 found
2	python -m venv mlops-env mlops-env\Scripts\activate	Environment created; prompt starts with (mlops-env)
3	python -m pip install --upgrade pip	pip upgraded from 24.3.1 to 26.2.2
4	pip install mlflow transformers torch pandas	mlflow 3.17.2, transformers 5.21.0, torch 2.15.0, pandas 3.1.0 and their dependencies
6	First pipeline() call for the model	About 250 MB of model files downloaded once and reused from cache afterwards

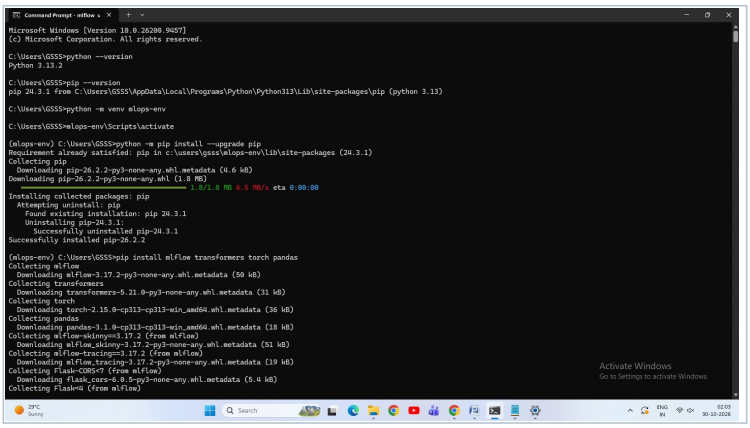


Figure 1: Python check, virtual environment, pip upgrade and package installation

3. Task 1: Tracked Experiment

The script **mlflow_lab.py** loads the classifier through the Hugging Face *pipeline* and runs it over the ten sentences (five positive, five negative). Inside *mlflow.start_run()* it logs three parameters (*model_name*, *batch_size*, *num_samples*), measures the inference time with *time.time()*, and logs three metrics: *accuracy*, *avg_confidence* and *total_latency_sec*. The predictions are saved to *predictions.csv* and logged as an artifact of the run. The full script, including the third run added for Task 3, is given below.

```
import time
import mlflow
import pandas as pd
from transformers import pipeline

# Small labelled dataset (10 sentences)
data = [
    ("I loved this movie, it was fantastic!", "POSITIVE"),
    ("The service was terrible and slow.", "NEGATIVE"),
    ("What a wonderful experience.", "POSITIVE"),
    ("I regret buying this product.", "NEGATIVE"),
    ("The food was delicious and fresh.", "POSITIVE"),
    ("This is the worst app I have ever used.", "NEGATIVE"),
    ("Absolutely brilliant performance by the team.", "POSITIVE"),
    ("The battery dies within an hour.", "NEGATIVE"),
    ("Great value for money, highly recommend.", "POSITIVE"),
    ("I am very disappointed with the quality.", "NEGATIVE"),
]
texts = [t for t, _ in data]
labels = [l for _, l in data]

mlflow.set_experiment("sentiment-inference-lab")

def run_experiment(model_name, batch_size, run_name):
    clf = pipeline("sentiment-analysis", model=model_name)
    with mlflow.start_run(run_name=run_name):
        # Parameters
        mlflow.log_param("model_name", model_name)
        mlflow.log_param("batch_size", batch_size)
        mlflow.log_param("num_samples", len(texts))

        # Inference
        start = time.time()
        preds = clf(texts, batch_size=batch_size)
        latency = time.time() - start

        pred_labels = [p["label"] for p in preds]
        accuracy = sum(p == l for p, l in zip(pred_labels, labels)) / len(labels)
        avg_conf = sum(p["score"] for p in preds) / len(preds)

        # Metrics
        mlflow.log_metric("accuracy", accuracy)
        mlflow.log_metric("avg_confidence", avg_conf)
        mlflow.log_metric("total_latency_sec", latency)
```

```
        # Artifact
        df = pd.DataFrame({
            "text": texts,
            "actual": labels,
            "predicted": pred_labels,
            "confidence": [p["score"] for p in preds],
        })
        df.to_csv("predictions.csv", index=False)
        mlflow.log_artifact("predictions.csv")

        print(run_name, "-> accuracy:", accuracy, "latency:", round(latency, 3))

MODEL = "distilbert-base-uncased-finetuned-sst-2-english"
run_experiment(MODEL, batch_size=1, run_name="run_batch1")
run_experiment(MODEL, batch_size=8, run_name="run_batch8")
run_experiment(MODEL, batch_size=4, run_name="run_batch4")   # Task 3
```

The file was created and edited in Notepad (Figure 2) and executed with **python mlflow_lab.py**. The terminal printed the accuracy and latency of each run, and **mlflow ui** then started the tracking server at http://127.0.0.1:5000 (Figure 3).

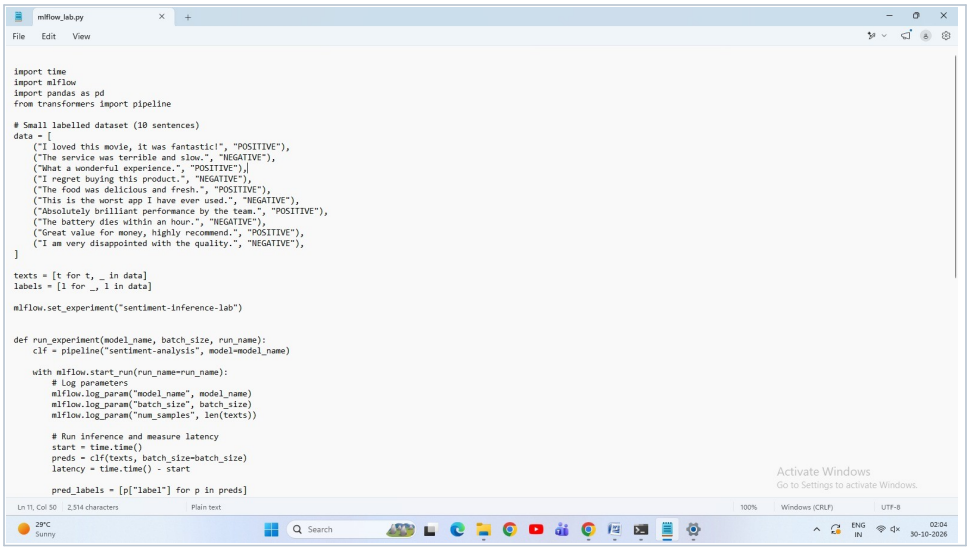


Figure 2: mlflow_lab.py open in Notepad

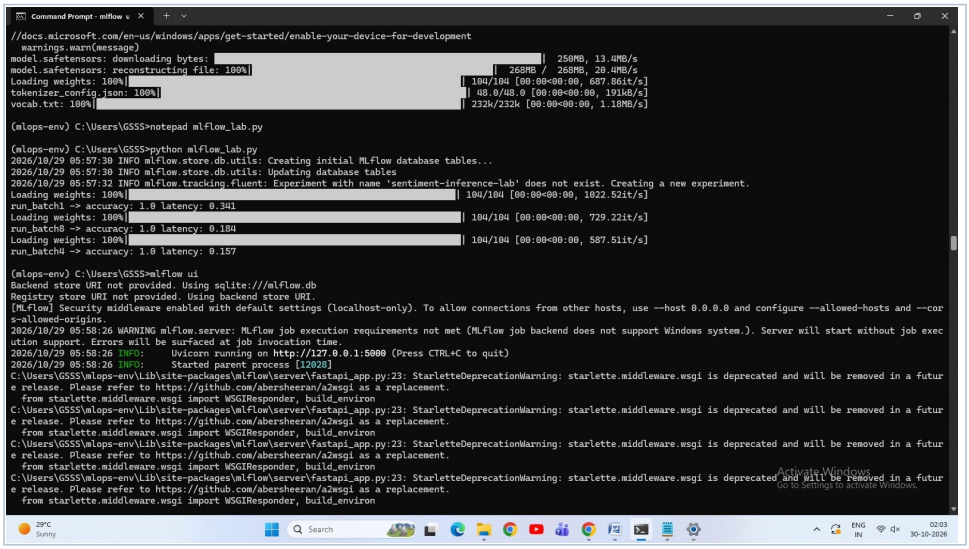


Figure 3: Terminal output of the three runs and start-up of the MLflow UI server

Results of the three runs (read from the MLflow UI):

Run	batch_size	accuracy	avg_confidence	total_latency_sec	Duration
run_batch1	1	1.0	0.9997	0.341	0.7 s
run_batch8	8	1.0	0.9997	0.184	345 ms
run_batch4	4	1.0	0.9997	0.157	238 ms

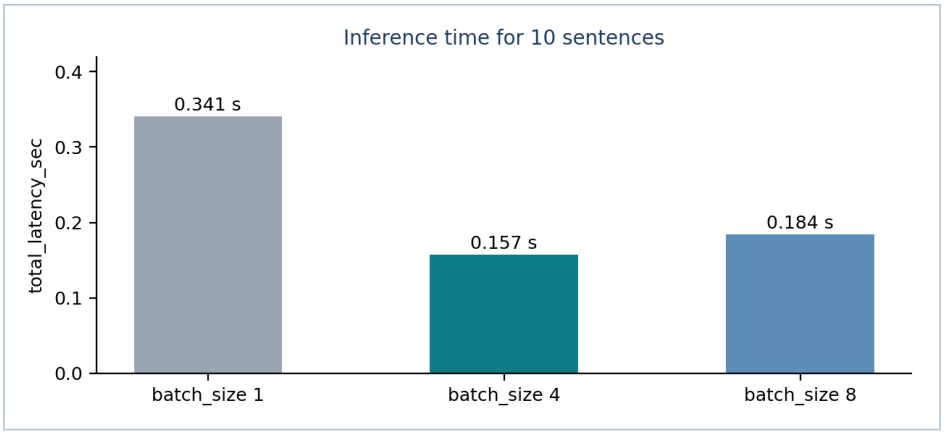


Figure 4: Total latency of each run (values taken from the logged metrics)

4. Task 2: Exploring the MLflow UI

After starting **mlflow ui**, I opened http://127.0.0.1:5000 in the browser and selected the experiment **sentiment-inference-lab**. All three runs are listed with status Finished (Figure 5). Each run also has its *predictions.csv* stored under the Artifacts tab.

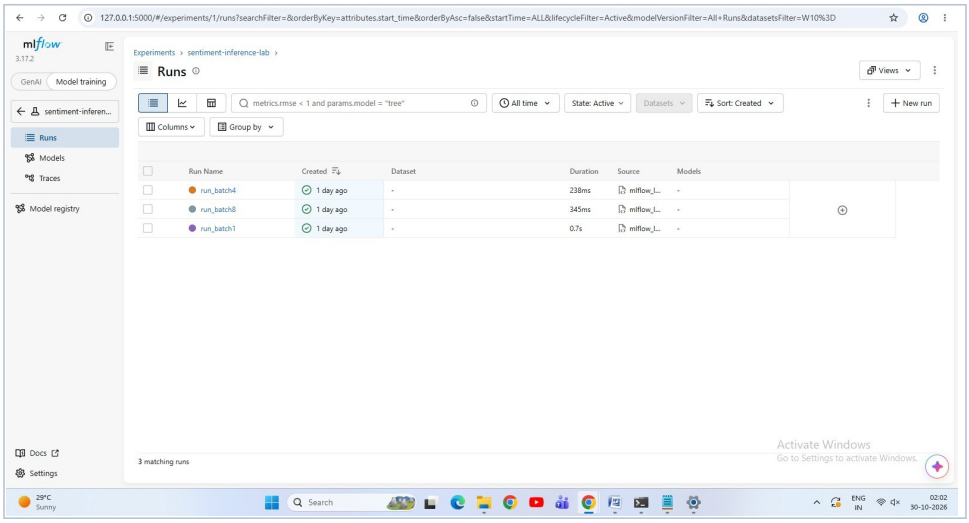


Figure 5: Runs table of sentiment-inference-lab showing the 3 matching runs

Next, all three runs were ticked and opened with the Compare option. The parallel coordinates plot joins *batch_size* on the left to *accuracy* on the right. Every line ends at the same accuracy value of 1.0, which shows at a glance that changing the batch size did not change the accuracy.

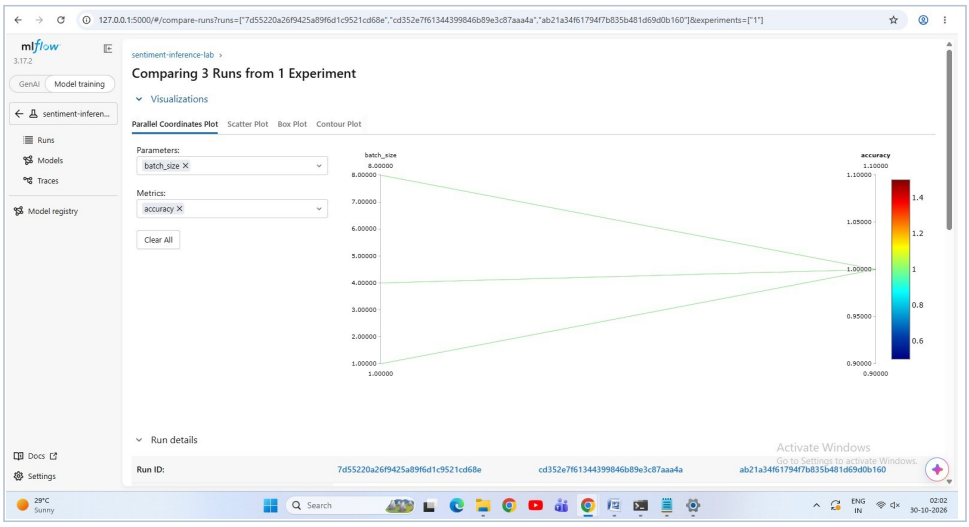


Figure 6: Compare view of the three runs: batch_size against accuracy

The individual run pages show the logged parameters and metrics. The pages of run_batch1 and run_batch8 are given below.

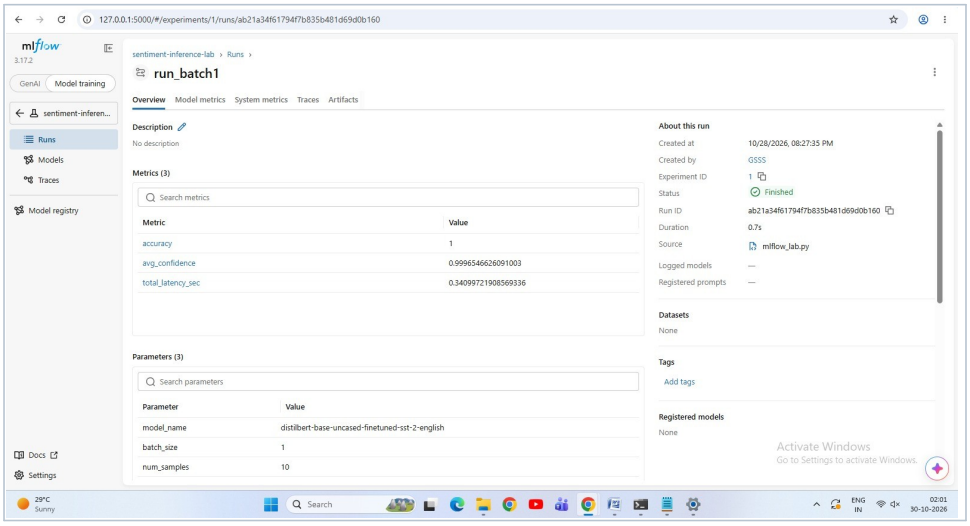


Figure 7: run_batch1 overview: metrics and parameters (batch_size = 1)

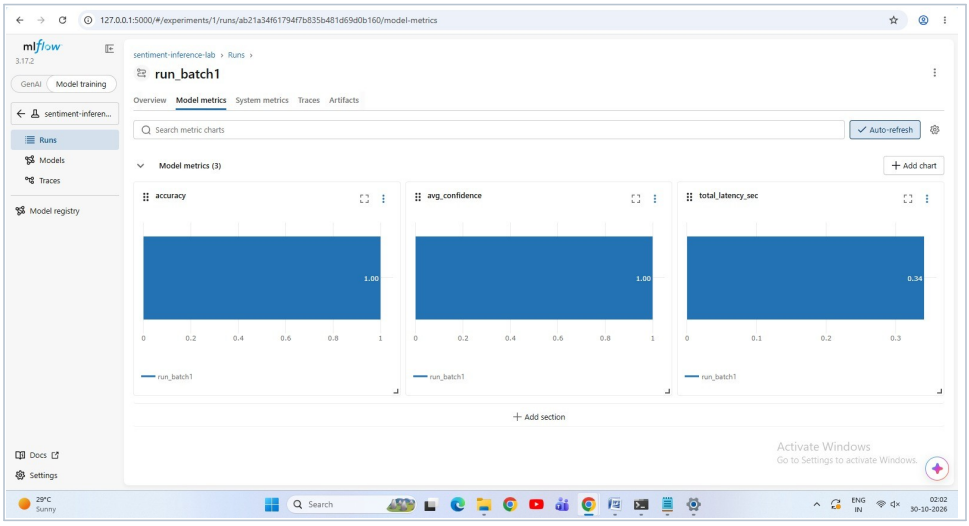


Figure 8: run_batch1 model metrics: accuracy 1.00, avg_confidence 1.00, total_latency_sec 0.34

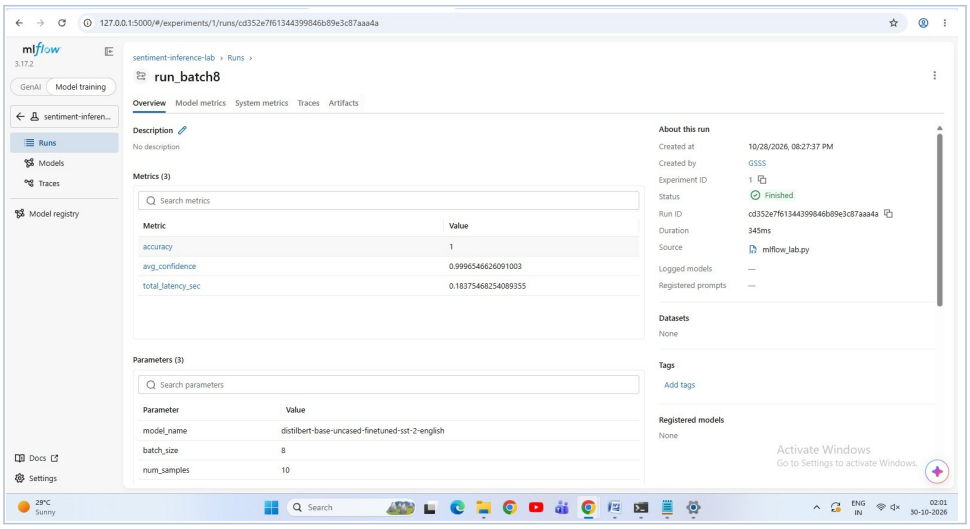


Figure 9: run_batch8 overview: metrics and parameters (batch_size = 8)

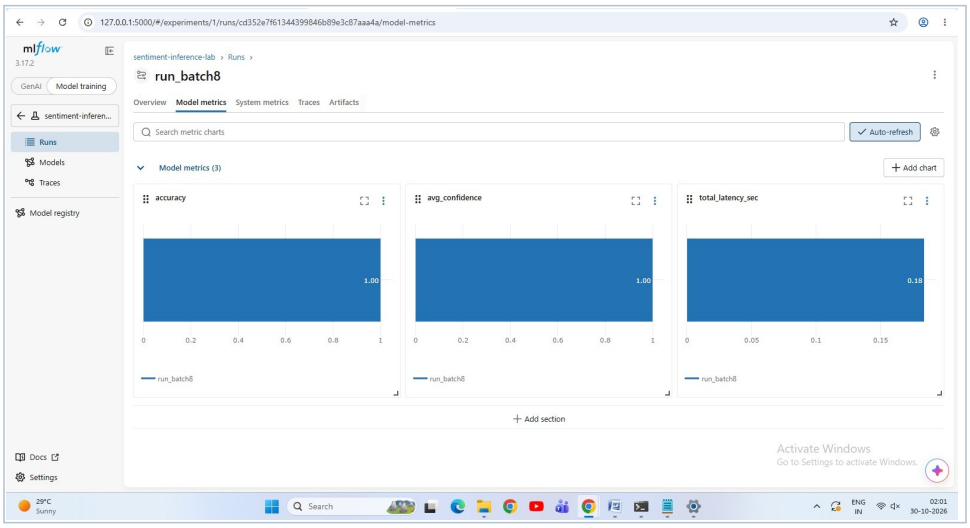


Figure 10: run_batch8 model metrics: accuracy 1.00, avg_confidence 1.00, total_latency_sec 0.18

5. Task 3: Third Experiment

For the third experiment I added one more call at the bottom of the script, using the same model with **batch_size=4** and the run name **run_batch4**. After running the script, the new run appeared in the MLflow UI next to the other two (top row of the runs table in Figure 5). Its overview and metrics pages are shown below.

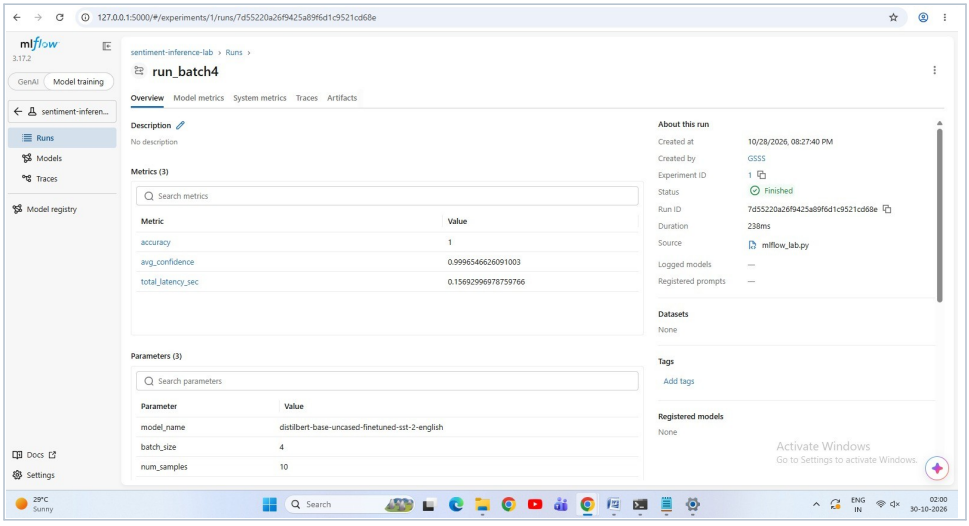


Figure 11: run_batch4 overview: metrics and parameters (batch_size = 4)

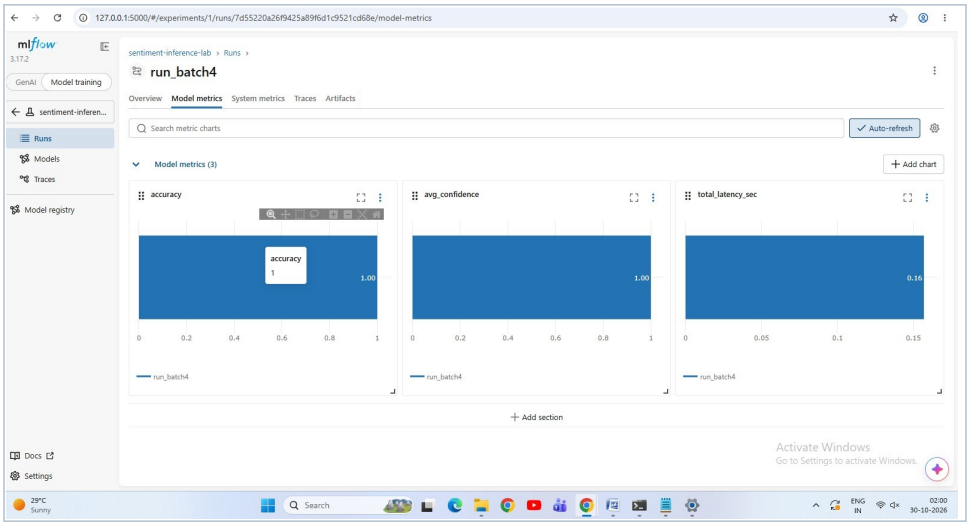


Figure 12: run_batch4 model metrics: accuracy 1.00, avg_confidence 1.00, total_latency_sec 0.16

6. Task 4: Short Report

1. **Lowest latency:** run_batch4 was the fastest at 0.157 s, followed by run_batch8 (0.184 s) and run_batch1 (0.341 s). With batching, the model handles several sentences in one pass, so less time is lost on repeated calls. Batch size 1 needs ten separate passes and, being the first run, probably also pays a start-up cost. The gap between 4 and 8 is small, so with only ten sentences it may be partly normal run-to-run variation.
2. **Accuracy:** It stayed at 1.0 in every run. The model, its weights and the ten sentences were the same each time, and batch size only changes how the inputs are grouped, not what the model predicts.
3. **Benefits of MLflow:** (i) Parameters, metrics and the predictions file are saved against each run ID, so any run can be reopened and checked later. (ii) The UI lets me compare runs side by side with tables and charts, which is easier than scrolling through printed terminal lines.

7. Submission Checklist

Item required	Where it is in this report
Screenshot of the MLflow runs table	Section 4, Figure 5
Screenshot of the Compare view	Section 4, Figure 6
mlflow_lab.py file	Section 3 (full code and Notepad screenshot)
Answers to the Task 4 questions	Section 6